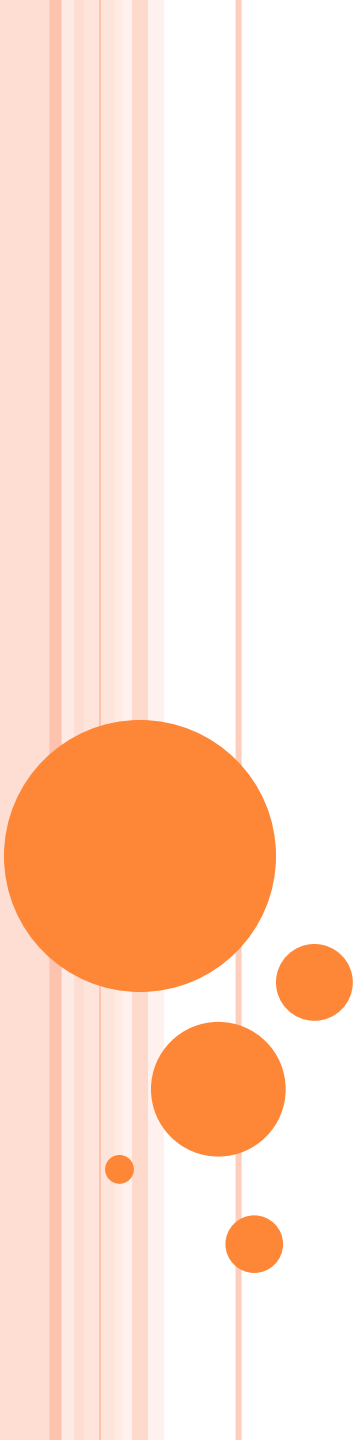




GRAPH THEORY

SINGLE SOURCE SHORTEST PATH

Design and Analysis of Algorithms

PATHS IN GRAPHS

Consider a digraph $G = (V, E)$ with edge-weight function $w : E \rightarrow \mathbb{R}$. The *weight* of path $p = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ is defined to be

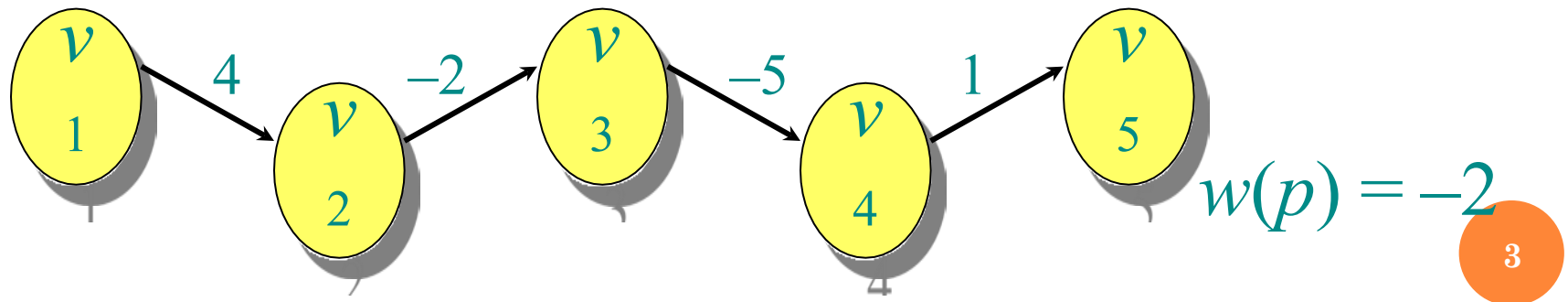
$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1}).$$

PATHS IN GRAPHS

Consider a digraph $G = (V, E)$ with edge-weight function $w : E \rightarrow \mathbb{R}$. The **weight** of path $p = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ is defined to be

$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1}).$$

Example:



SHORTEST PATHS

A *shortest path* from u to v is a path of minimum weight from u to v . The *shortest-path weight* from u to v is defined as

$$\delta(u, v) = \min \{w(p) : p \text{ is a path from } u \text{ to } v\}.$$

Note: $\delta(u, v) = \infty$ if no path from u to v exists.

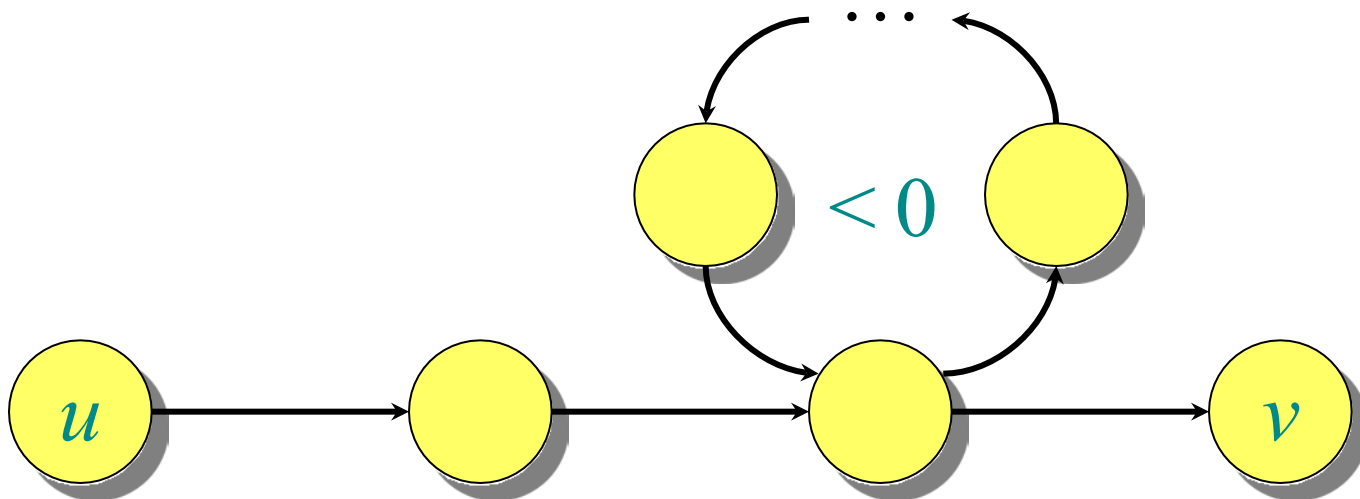
If a graph G contains a negative-weight cycle, then some shortest paths do not exist.

WELL-DEFINEDNESS OF SHORTEST PATHS

WELL-DEFINEDNESS OF SHORTEST PATHS

If a graph G contains a negative-weight cycle, then some shortest paths do not exist.

Example:



NEGATIVE WEIGHT-REAL WORLD APPLICATIONS

The weight on edges can represent anything in the real world, for example

- Finance:

- Arbitrage: Performing an arbitrage (the simultaneous buying and selling of securities, currency, or commodities in different markets or in derivative forms in order to take advantage of differing prices for the same asset between two or more markets)
- Money Transfer: the amount of money to be transferred from one account to another account. The amounts can be positive or negative. For example, if you want to go from a to b in your graph while losing as less money as possible (shortest path), then you can consider negative weights.

- Chemistry

- The weights can be used to represent the heat produced during a chemical reaction.

NEGATIVE-WEIGHT CYCLE

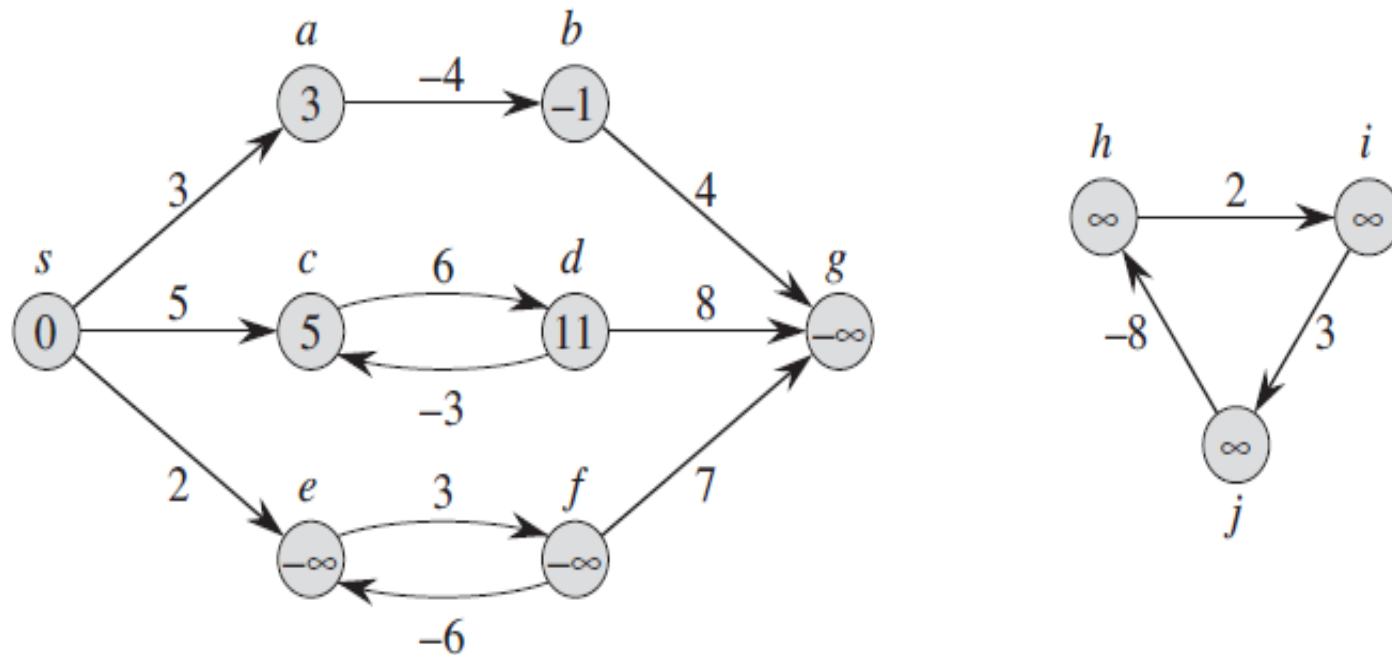
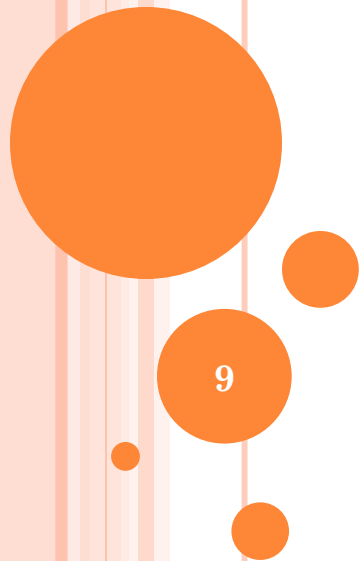


Figure 24.1 Negative edge weights in a directed graph. The shortest-path weight from source s appears within each vertex. Because vertices e and f form a negative-weight cycle reachable from s , they have shortest-path weights of $-\infty$. Because vertex g is reachable from a vertex whose shortest-path weight is $-\infty$, it, too, has a shortest-path weight of $-\infty$. Vertices such as h, i , and j are not reachable from s , and so their shortest-path weights are ∞ , even though they lie on a negative-weight cycle.

Optimal substructure

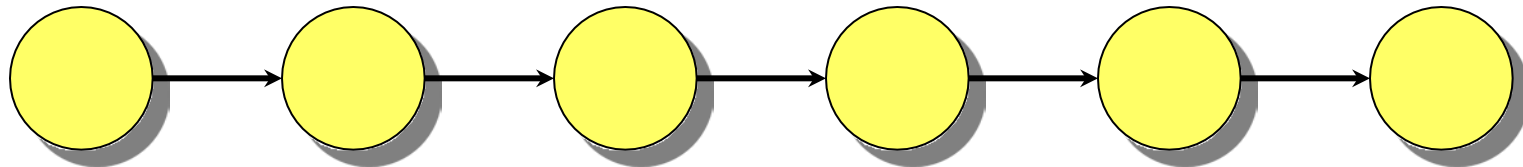
Theorem. A subpath of a shortest path is a shortest path.



OPTIMAL SUBSTRUCTURE

Theorem. A subpath of a shortest path is a shortest path.

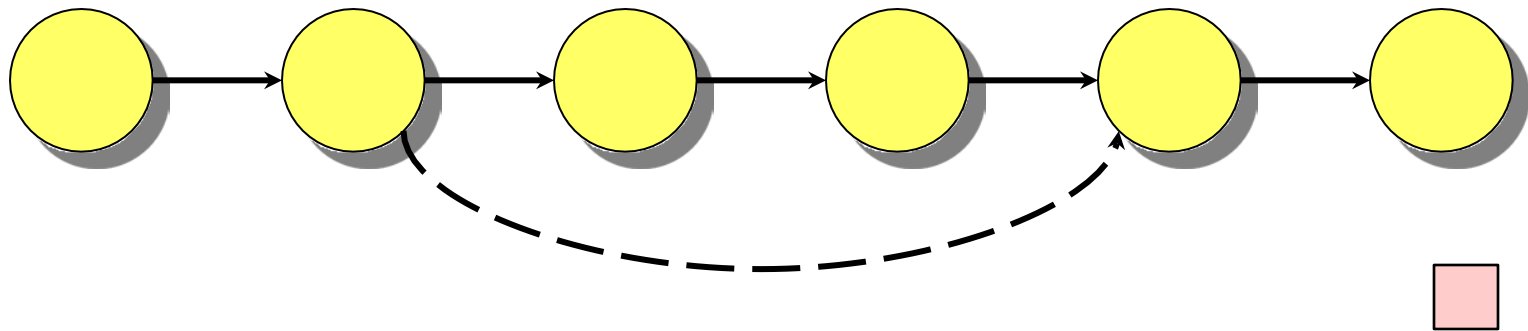
Proof. Cut and paste:



OPTIMAL SUBSTRUCTURE

Theorem. A subpath of a shortest path is a shortest path.

Proof. Cut and paste:



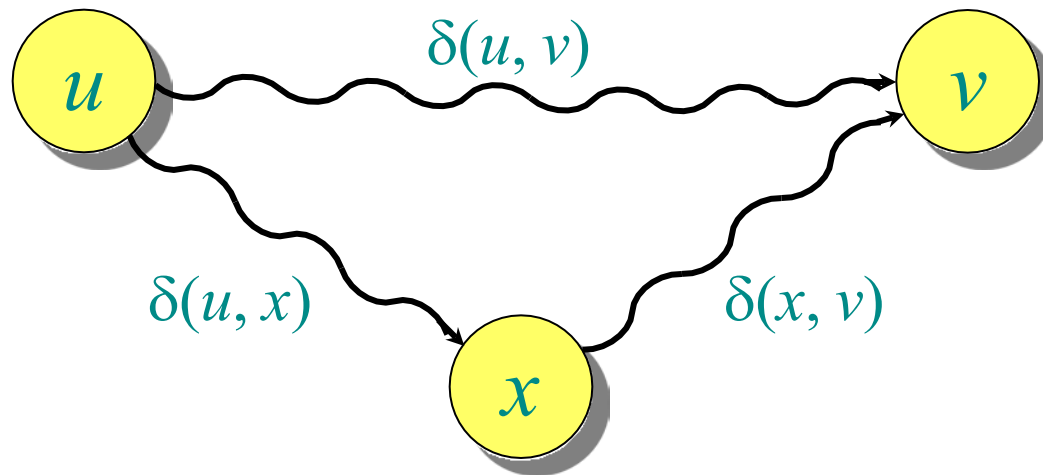
TRIANGLE INEQUALITY

Theorem. For all $u, v, x \in V$, we have
$$\delta(u, v) \leq \delta(u, x) + \delta(x, v).$$

TRIANGLE INEQUALITY

Theorem. For all $u, v, x \in V$, we have
$$\delta(u, v) \leq \delta(u, x) + \delta(x, v).$$

Proof.



SINGLE-SOURCE SHORTEST PATHS (NONNEGATIVE EDGE WEIGHTS)

Problem. Assume that $w(u, v) \geq 0$ for all $(u, v) \in E$. (Hence, all shortest-path weights must exist.) From a given source vertex $s \in V$, find the shortest-path weights $\delta(s, v)$ for all $v \in V$.

IDEA: Greedy.

1. Maintain a set S of vertices whose shortest-path distances from s are known.
2. At each step, add to S the vertex $v \in V - S$ whose distance estimate from s is minimum.
3. Update the distance estimates of vertices adjacent to v .

DIJKSTRA'S ALGORITHM

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$

do $d[v] \leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$

▷ Q is a priority queue maintaining $V - S$,
keyed on $d[v]$

DIJKSTRA'S ALGORITHM

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$

do $d[v] \leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$

 ▷ Q is a priority queue maintaining $V - S$,
 keyed on $d[v]$

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

for each $v \in \text{Adj}[u]$

do if $d[v] > d[u] + w(u, v)$

then $d[v] \leftarrow d[u] + w(u, v)$

DIJKSTRA'S ALGORITHM

$d[s] \leftarrow 0$

for each $v \in V - \{s\}$

do $d[v] \leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$

▷ Q is a priority queue maintaining $V - S$,
keyed on $d[v]$

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

for each $v \in \text{Adj}[u]$

do if $d[v] > d[u] + w(u, v)$

then $d[v] \leftarrow d[u] + w(u, v)$

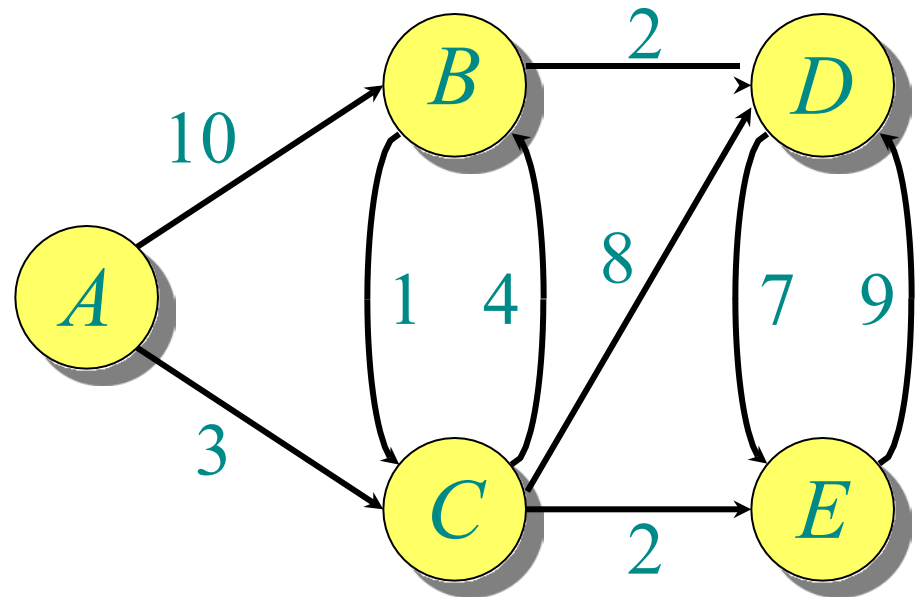
*relaxation
step*

↑ Implicit DECREASE-KEY

17

EXAMPLE OF DIJKSTRA'S ALGORITHM

**Graph with
nonnegative
edge weights:**

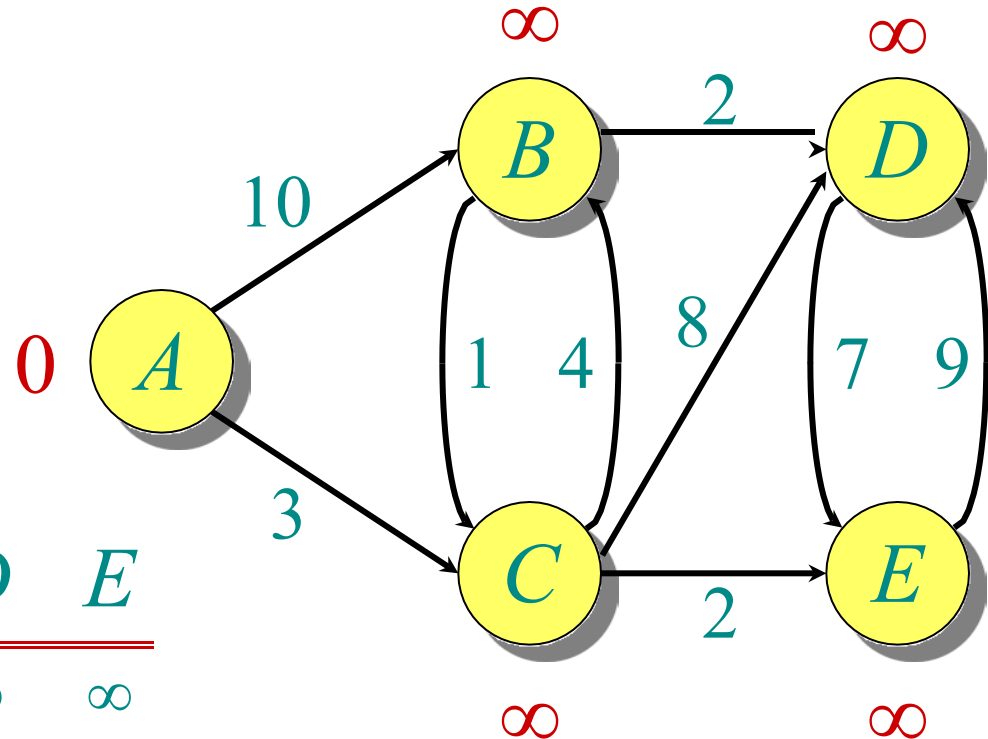


EXAMPLE OF DIJKSTRA'S ALGORITHM

Initialize:

$Q:$

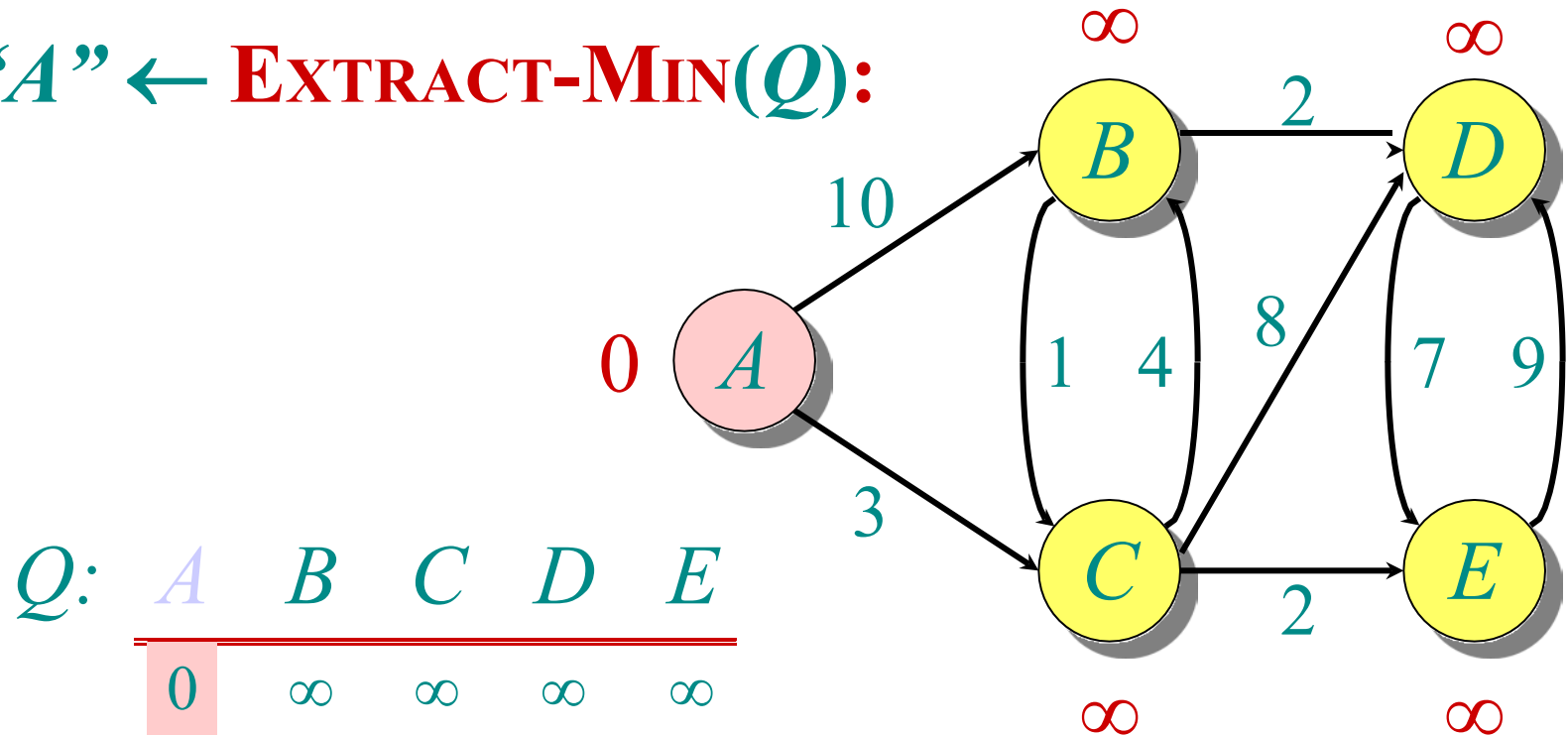
A	B	C	D	E
0	∞	∞	∞	∞



$S: \{\}$

EXAMPLE OF DIJKSTRA'S ALGORITHM

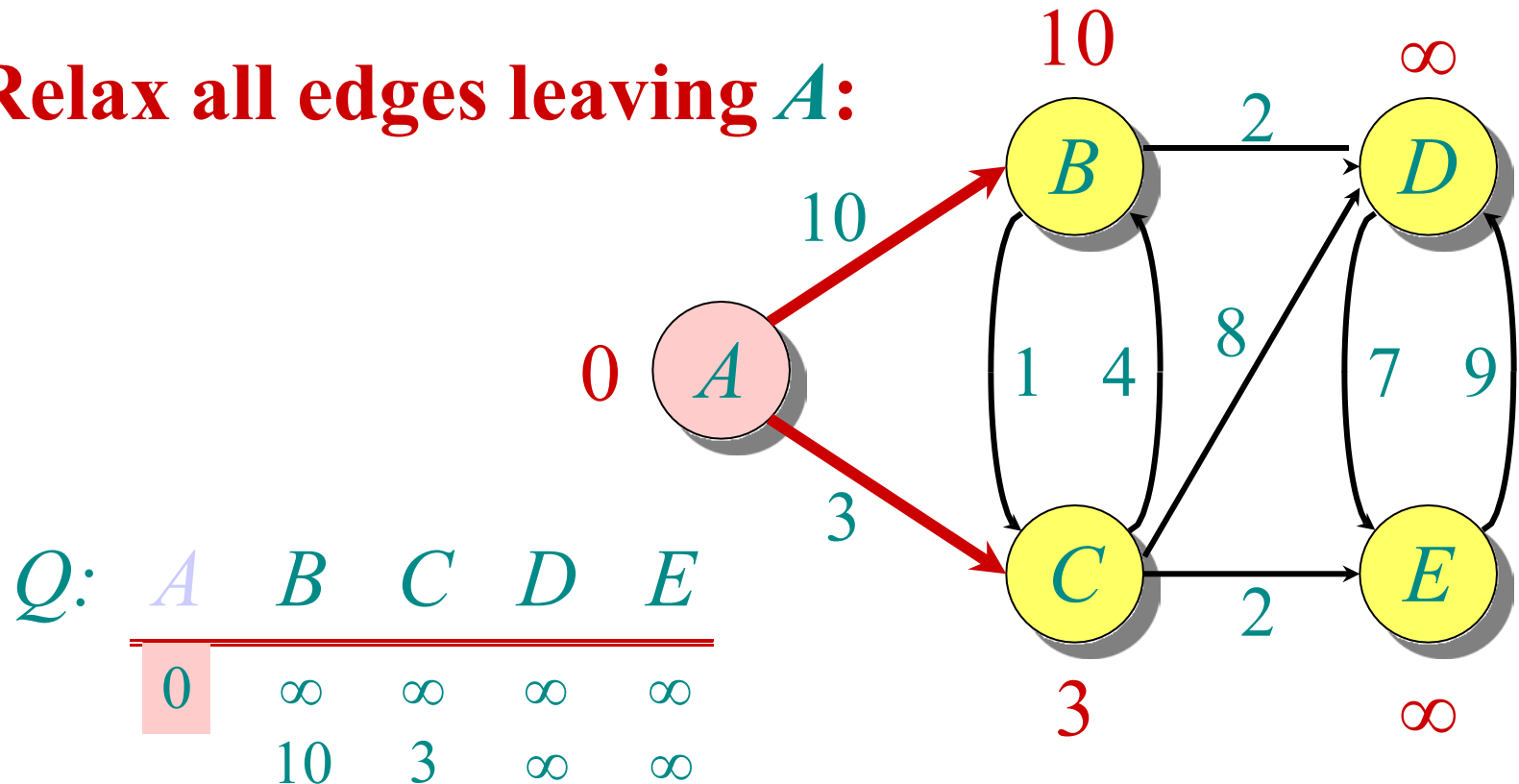
“A” $\leftarrow$ **EXTRACT-MIN**(Q):



$S: \{ A \}$

EXAMPLE OF DIJKSTRA'S ALGORITHM

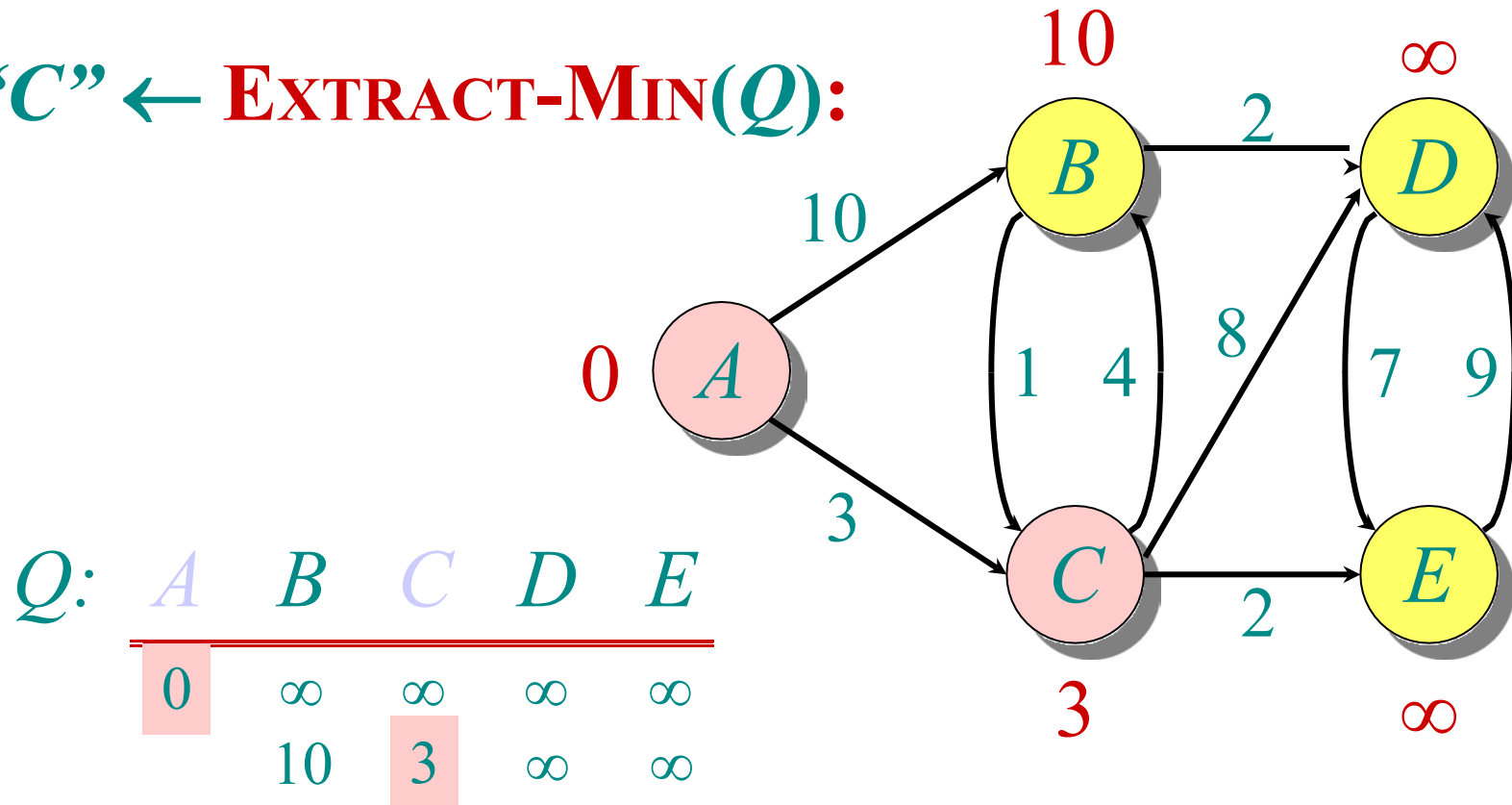
Relax all edges leaving A :



$S: \{ A \}$

EXAMPLE OF DIJKSTRA'S ALGORITHM

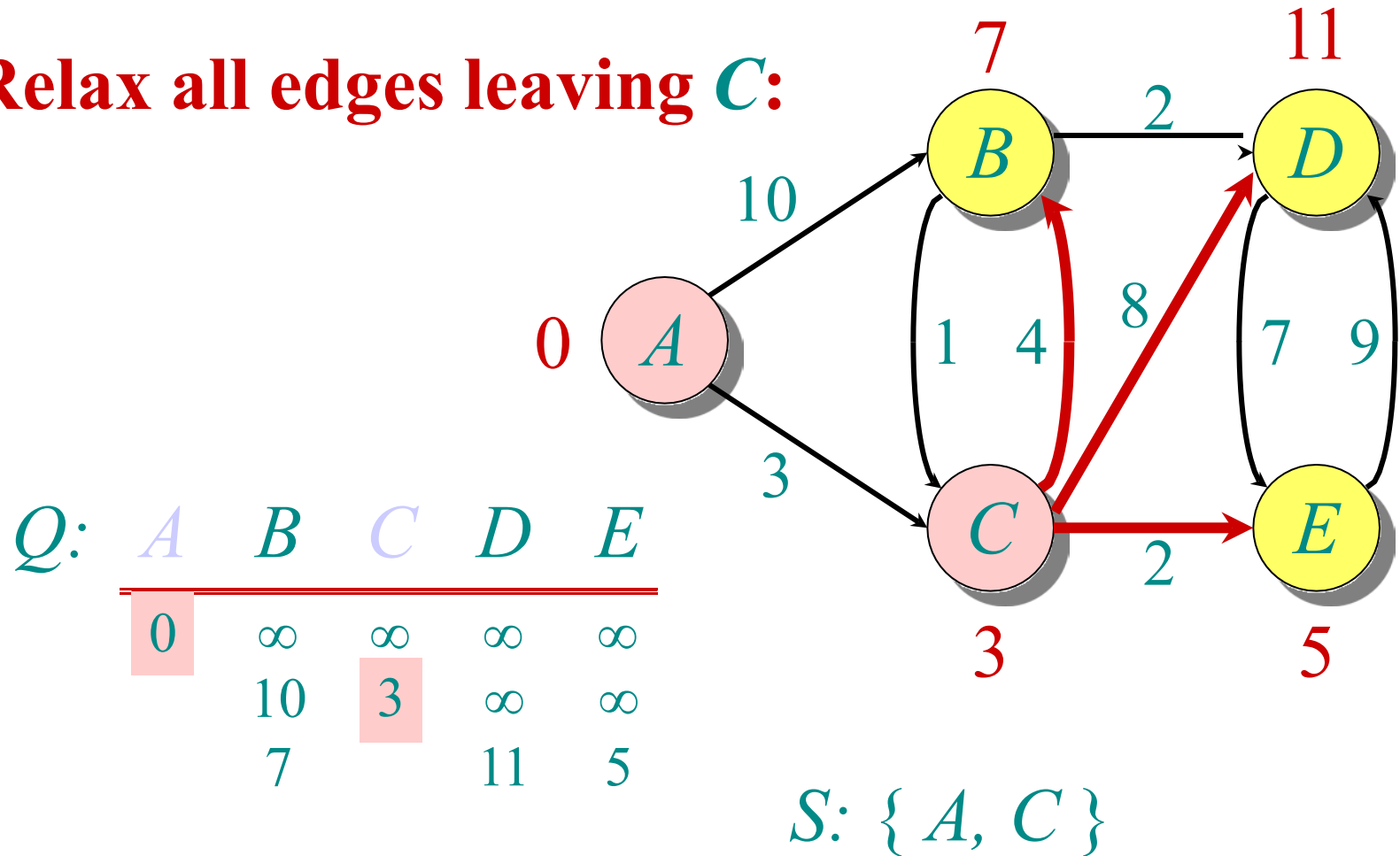
“C” $\leftarrow$ **EXTRACT-MIN**(*Q*):



S: { *A*, *C* }

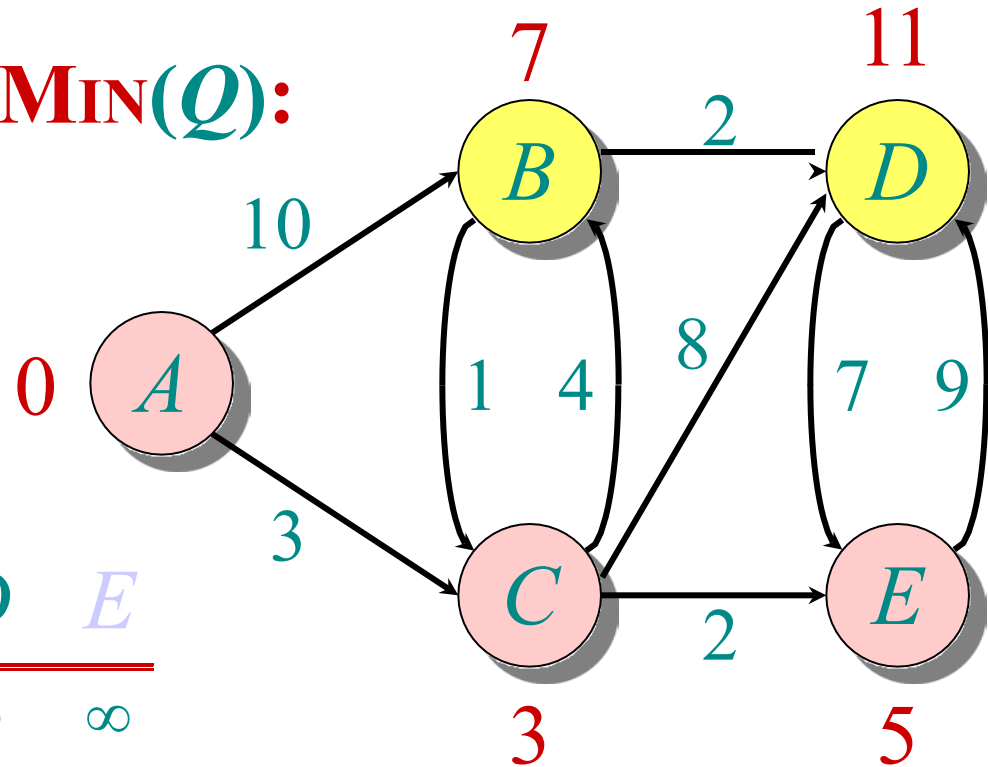
EXAMPLE OF DIJKSTRA'S ALGORITHM

Relax all edges leaving **C**:



EXAMPLE OF DIJKSTRA'S ALGORITHM

“E” $\leftarrow$ **EXTRACT-MIN**(*Q*):



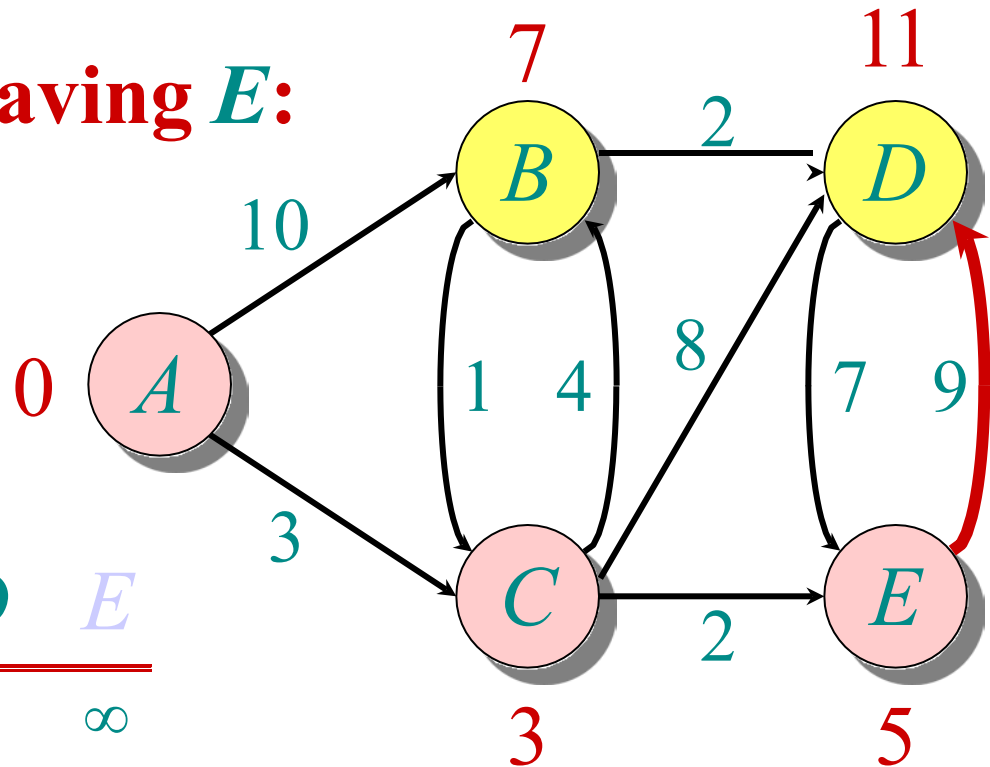
Q:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
	10	3	∞	∞
	7		11	5

S: { *A*, *C*, *E* }

EXAMPLE OF DIJKSTRA'S ALGORITHM

Relax all edges leaving *E*:



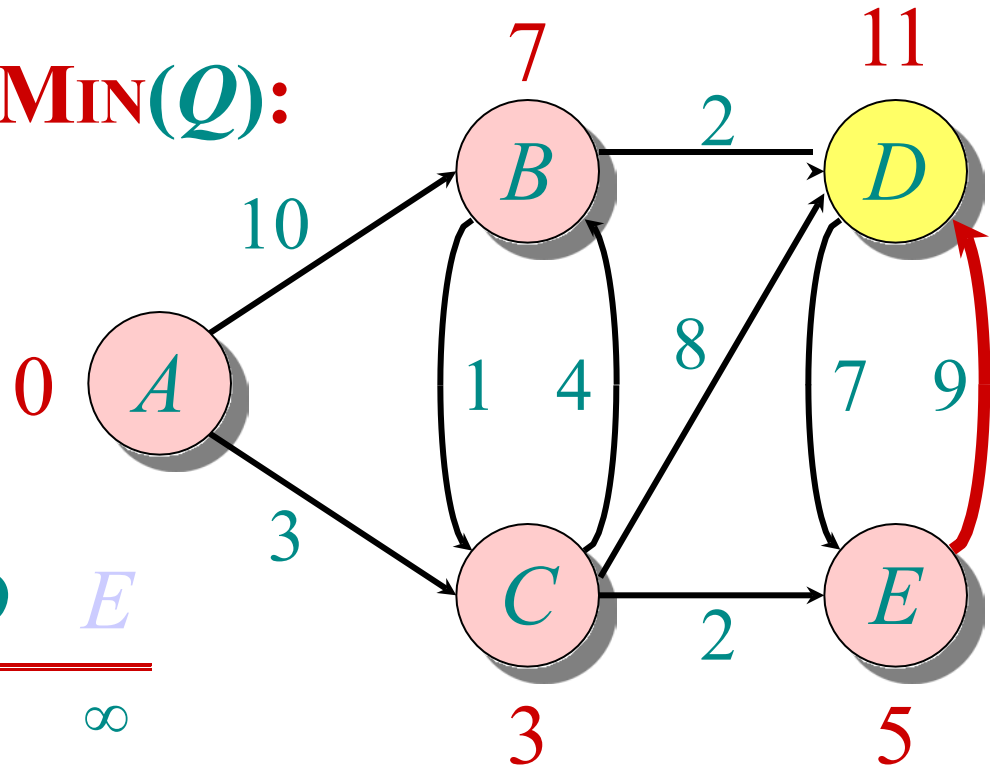
Q:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
	10	3	∞	∞
	7		11	5
	7		11	

S: { *A*, *C*, *E* }

EXAMPLE OF DIJKSTRA'S ALGORITHM

“B” $\leftarrow$ **EXTRACT-MIN**(*Q*):



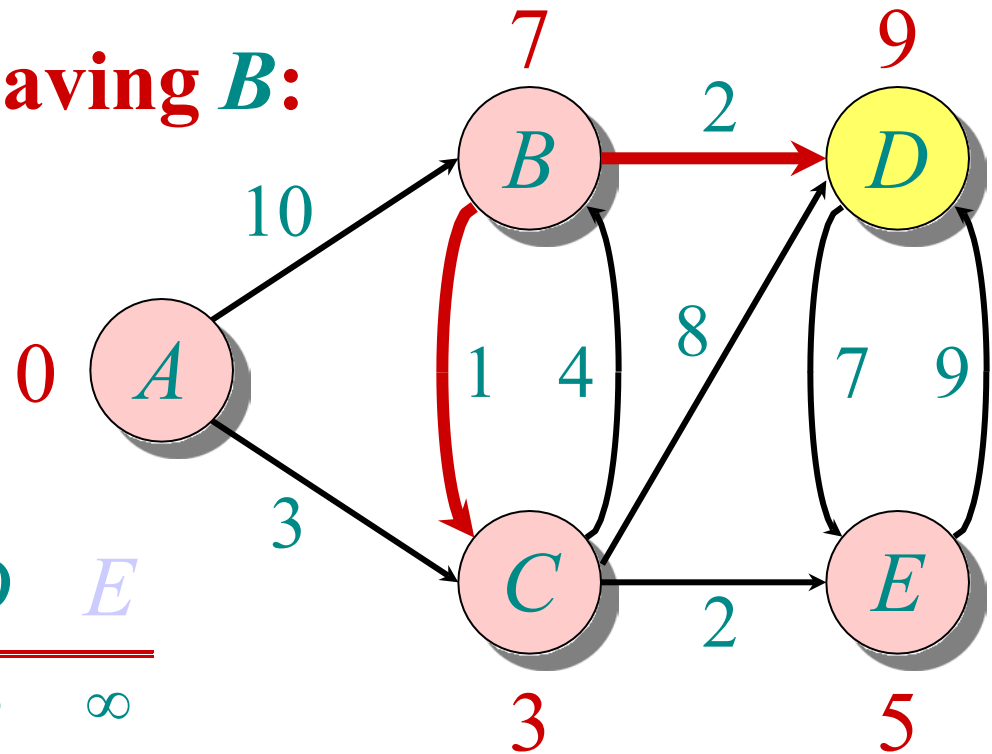
Q:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
	10	3	∞	∞
	7		11	5
	7		11	

S: { *A*, *C*, *E*, *B* }

EXAMPLE OF DIJKSTRA'S ALGORITHM

Relax all edges leaving *B*:



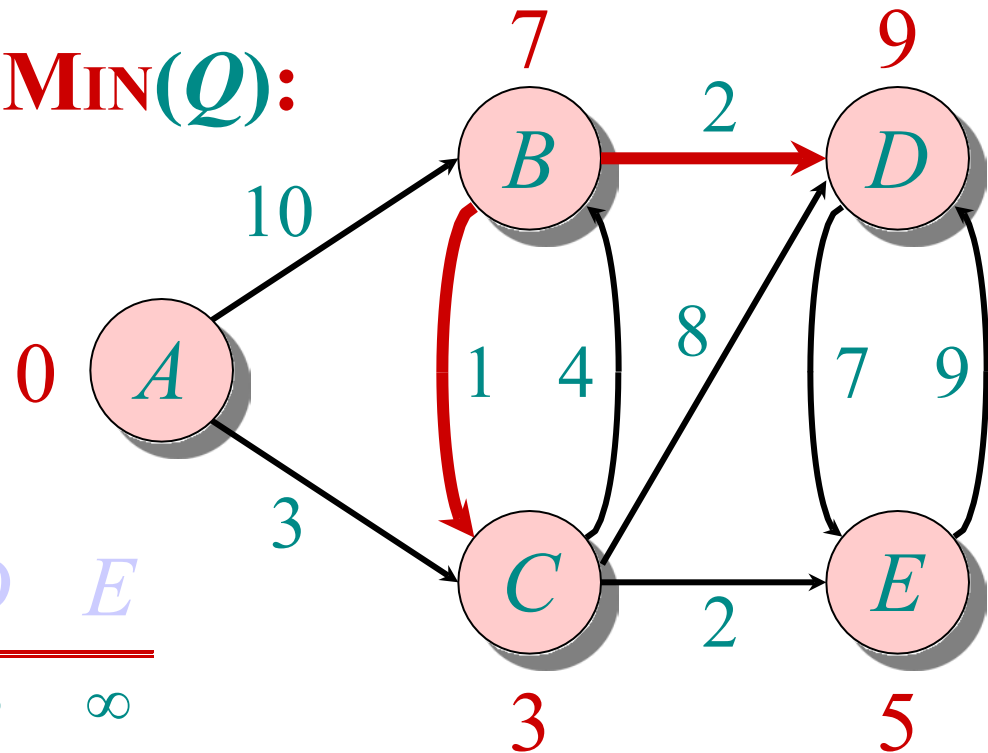
Q:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
	10	3	∞	∞
	7		11	5
	7		11	
			9	

S: { *A*, *C*, *E*, *B* }

EXAMPLE OF DIJKSTRA'S ALGORITHM

“D” $\leftarrow$ **EXTRACT-MIN(Q):**



Q:

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
0	∞	∞	∞	∞
	10	3	∞	∞
	7		11	5
	7		11	
			9	

S: { *A*, *C*, *E*, *B*, *D* }

ANALYSIS OF DIJKSTRA

```
while  $Q \neq \emptyset$   
  do  $u \leftarrow \text{EXTRACT-MIN}(Q)$   
     $S \leftarrow S \cup \{u\}$   
    for each  $v \in \text{Adj}[u]$   
      do if  $d[v] > d[u] + w(u, v)$   
        then  $d[v] \leftarrow d[u] + w(u, v)$ 
```

ANALYSIS OF DIJKSTRA

$|V|$
times {

```
while  $Q \neq \emptyset$   
  do  $u \leftarrow \text{EXTRACT-MIN}(Q)$   
     $S \leftarrow S \cup \{u\}$   
    for each  $v \in \text{Adj}[u]$   
      do if  $d[v] > d[u] + w(u, v)$   
        then  $d[v] \leftarrow d[u] + w(u, v)$ 
```

ANALYSIS OF DIJKSTRA

$|V|$
times { **while** $Q \neq \emptyset$
 do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 $S \leftarrow S \cup \{u\}$
 for each $v \in \text{Adj}[u]$
 do if $d[v] > d[u] + w(u, v)$
 then $d[v] \leftarrow d[u] + w(u, v)$

$\text{degree}(u)$
times {

ANALYSIS OF DIJKSTRA

$|V|$ times { **while** $Q \neq \emptyset$
do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 $S \leftarrow S \cup \{u\}$
 $\text{degree}(u)$ times { **for each** $v \in \text{Adj}[u]$
do **if** $d[v] > d[u] + w(u, v)$
then $d[v] \leftarrow d[u] + w(u, v)$

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

ANALYSIS OF DIJKSTRA

$|V|$ times { **while** $Q \neq \emptyset$
do $u \leftarrow \text{EXTRACT-MIN}(Q)$
 $S \leftarrow S \cup \{u\}$
degree(u) times { **for each** $v \in \text{Adj}[u]$
do **if** $d[v] > d[u] + w(u, v)$
then $d[v] \leftarrow d[u] + w(u, v)$

Handshaking Lemma $\Rightarrow \Theta(E)$ implicit DECREASE-KEY's.

$$\text{Time} = \Theta(V \cdot T_{\text{EXTRACT-MIN}} + E \cdot T_{\text{DECREASE-KEY}})$$

Note: Same formula as in the analysis of Prim's minimum spanning tree algorithm.

ANALYSIS OF DIJKSTRA (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
-----	--------------------------	---------------------------	-------

ANALYSIS OF DIJKSTRA (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
-----	--------------------------	---------------------------	-------

array	$O(V)$	$O(1)$	$O(V^2)$
-------	--------	--------	----------

ANALYSIS OF DIJKSTRA (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
-----	--------------------------	---------------------------	-------

array	$O(V)$	$O(1)$	$O(V^2)$
-------	--------	--------	----------

binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
----------------	------------	------------	--------------

ANALYSIS OF DIJKSTRA (CONTINUED)

$$\text{Time} = \Theta(V) \cdot T_{\text{EXTRACT-MIN}} + \Theta(E) \cdot T_{\text{DECREASE-KEY}}$$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$ amortized	$O(1)$ amortized	$O(E + V \lg V)$ worst case

Unweighted graphs

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.
Can Dijkstra's algorithm be improved?

UNWEIGHTED GRAPHS

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.

Can Dijkstra's algorithm be improved?

- Use a simple FIFO queue instead of a priority queue.

UNWEIGHTED GRAPHS

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.

Can Dijkstra's algorithm be improved?

- Use a simple FIFO queue instead of a priority queue.

Breadth-first search

```
while  $Q \neq \emptyset$ 
  do  $u \leftarrow \text{DEQUEUE}(Q)$ 
    for each  $v \in \text{Adj}[u]$ 
      do if  $d[v] = \infty$ 
        then  $d[v] \leftarrow d[u] + 1$ 
          ENQUEUE( $Q, v$ )
```


UNWEIGHTED GRAPHS

Suppose that $w(u, v) = 1$ for all $(u, v) \in E$.

Can Dijkstra's algorithm be improved?

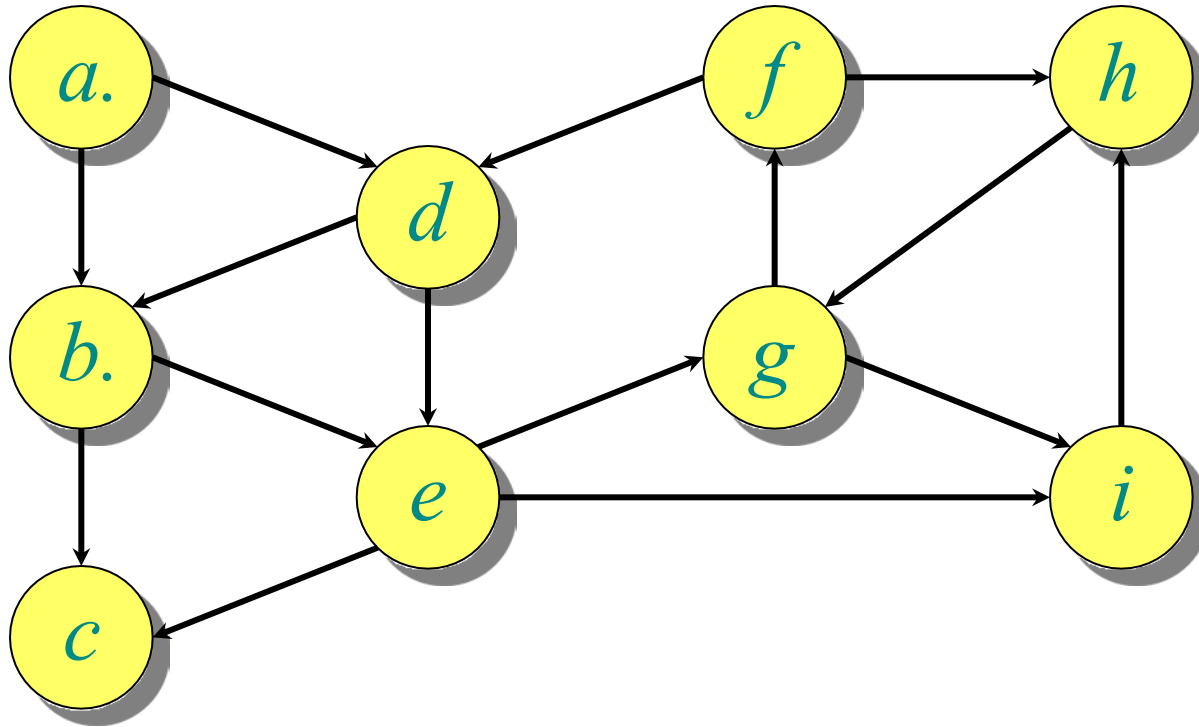
- Use a simple FIFO queue instead of a priority queue.

Breadth-first search

```
while  $Q \neq \emptyset$ 
  do  $u \leftarrow \text{DEQUEUE}(Q)$ 
    for each  $v \in \text{Adj}[u]$ 
      do if  $d[v] = \infty$ 
        then  $d[v] \leftarrow d[u] + 1$ 
          ENQUEUE( $Q, v$ )
```

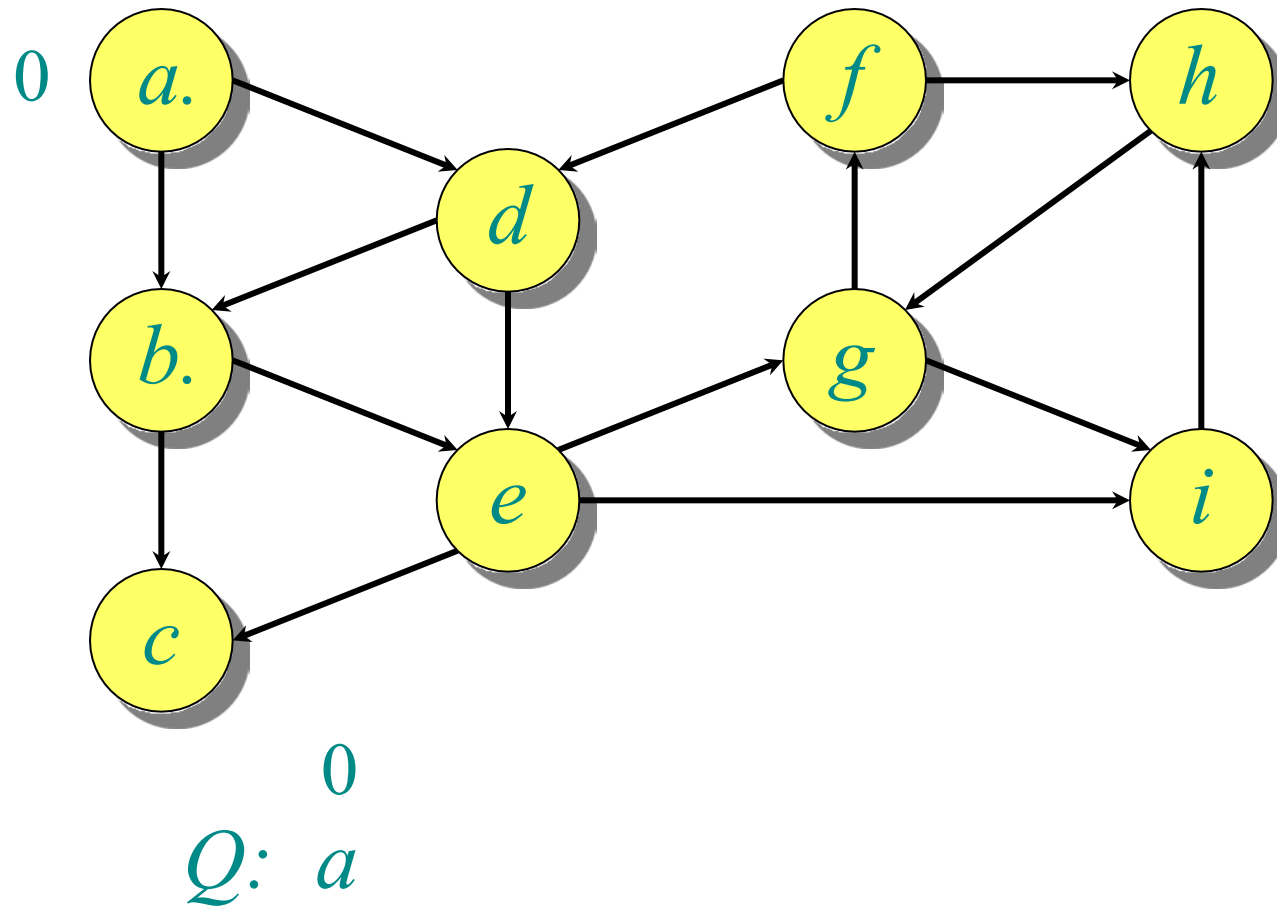
Analysis: Time = $O(V + E)$.

EXAMPLE OF BREADTH-FIRST SEARCH

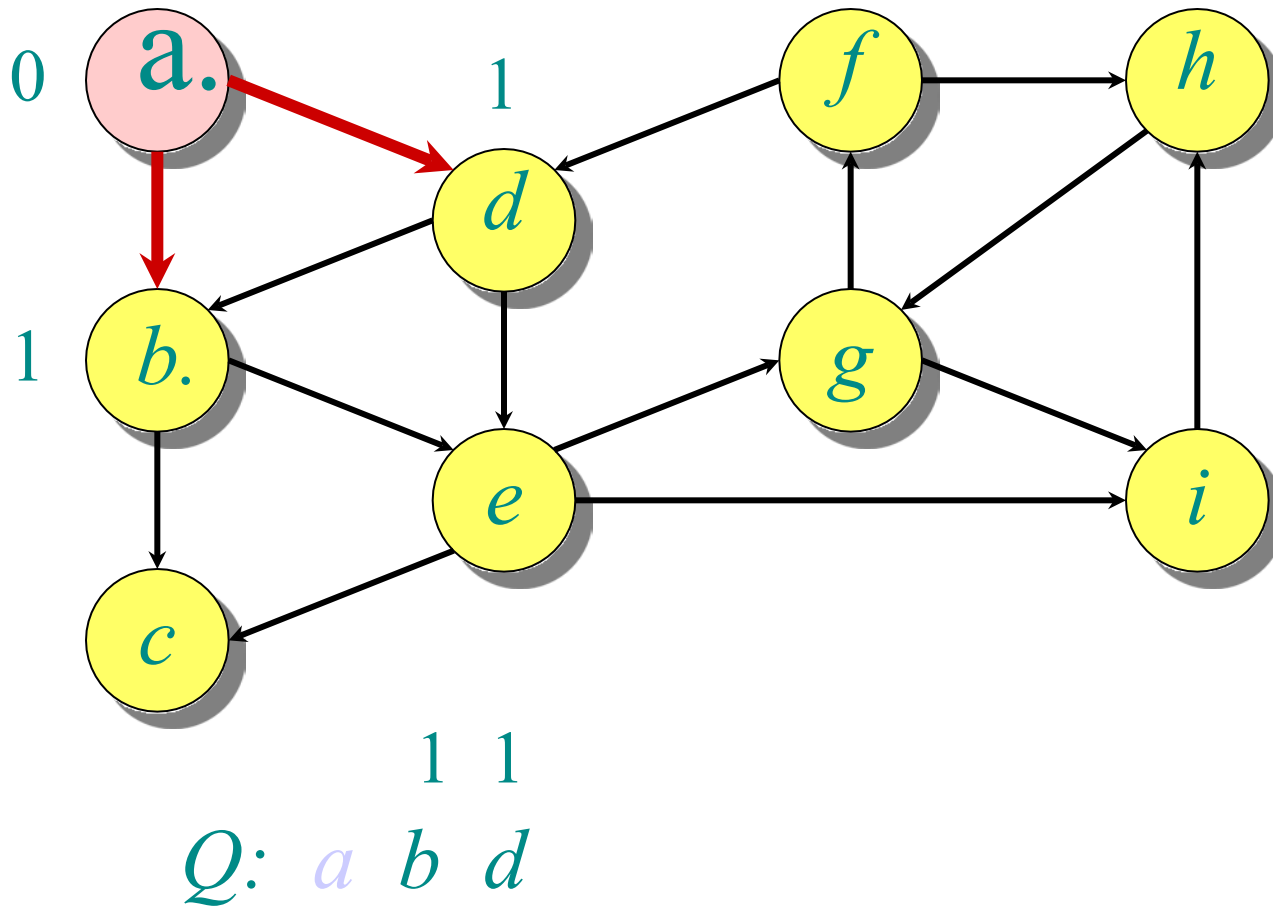


Q:

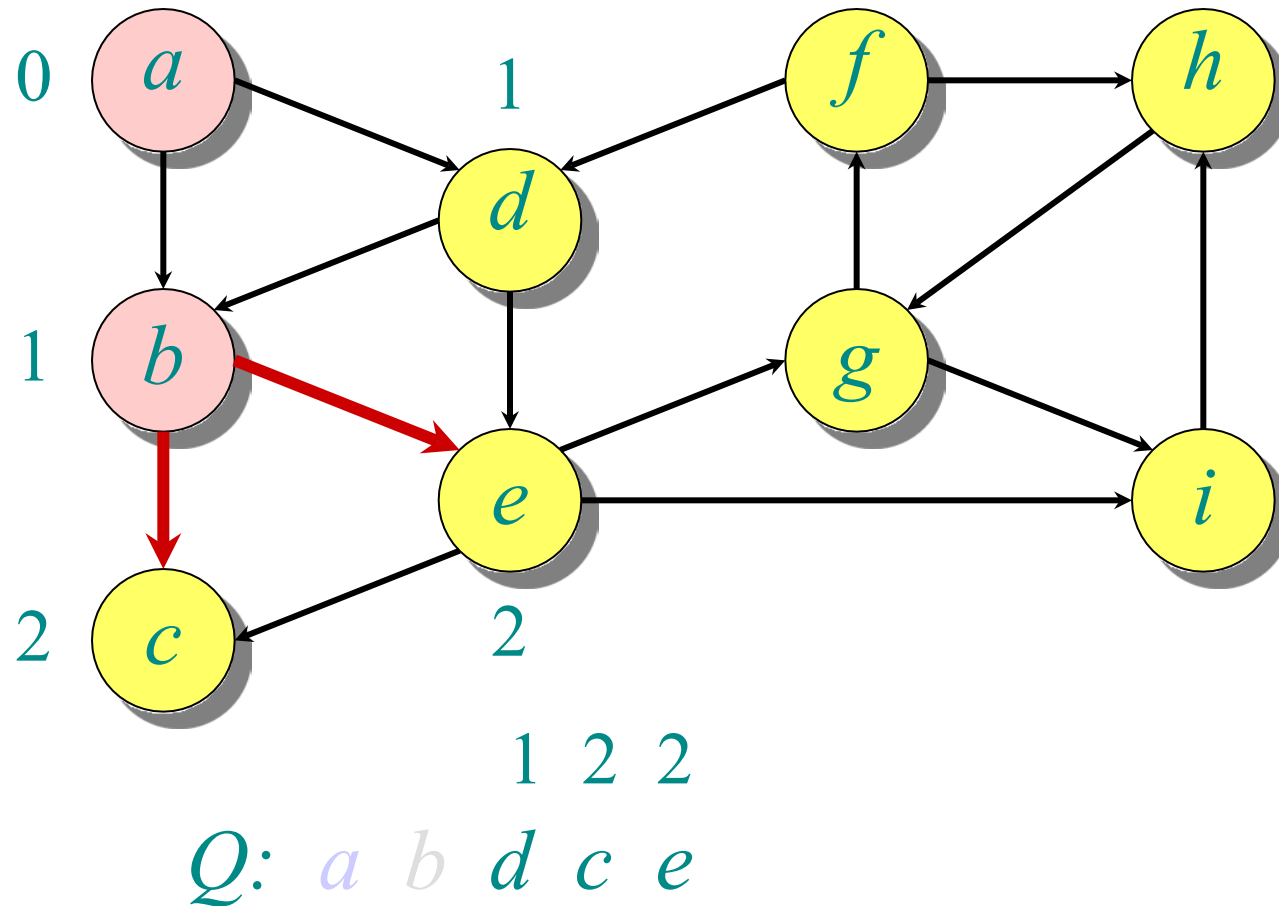
EXAMPLE OF BREADTH-FIRST SEARCH



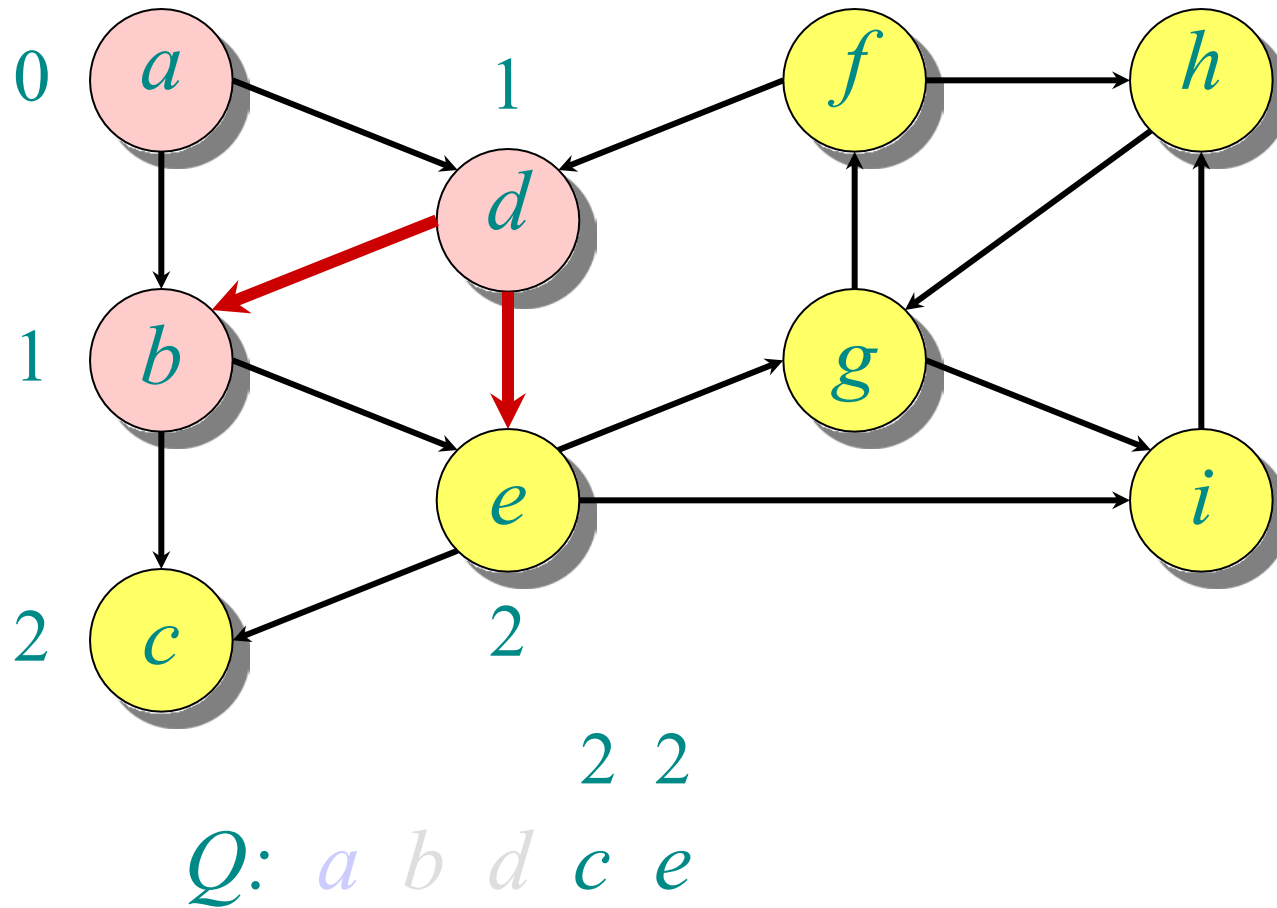
EXAMPLE OF BREADTH-FIRST SEARCH



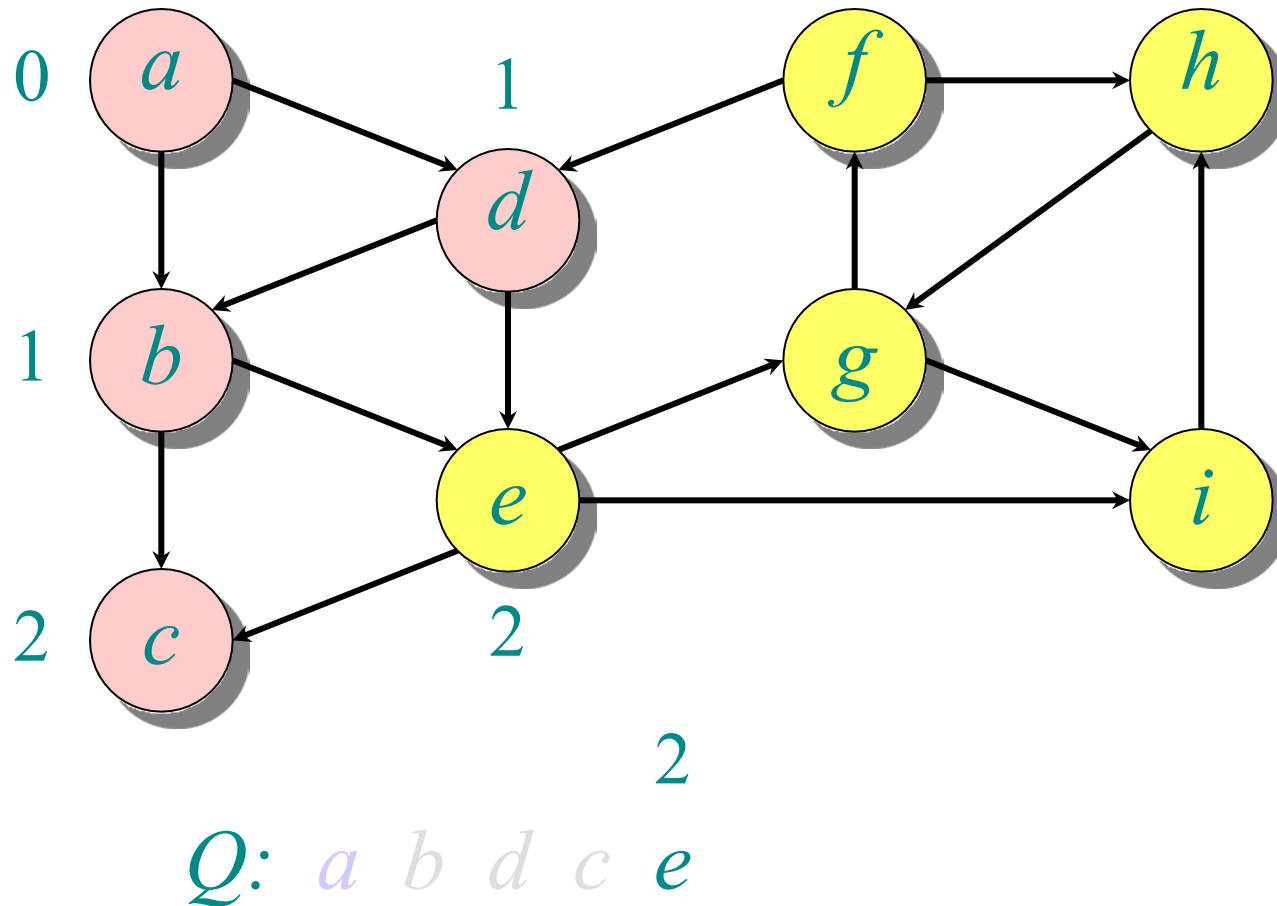
EXAMPLE OF BREADTH-FIRST SEARCH



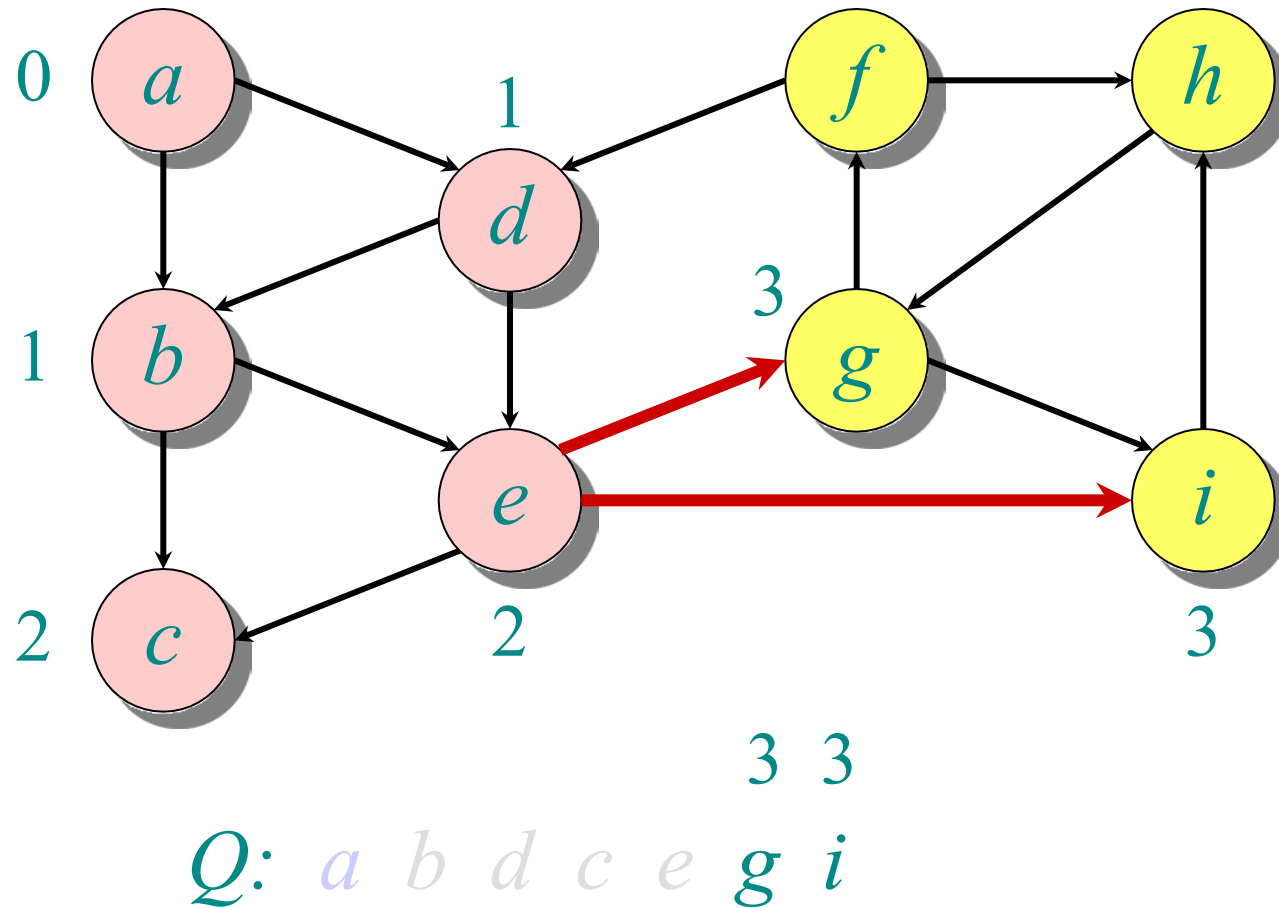
EXAMPLE OF BREADTH-FIRST SEARCH



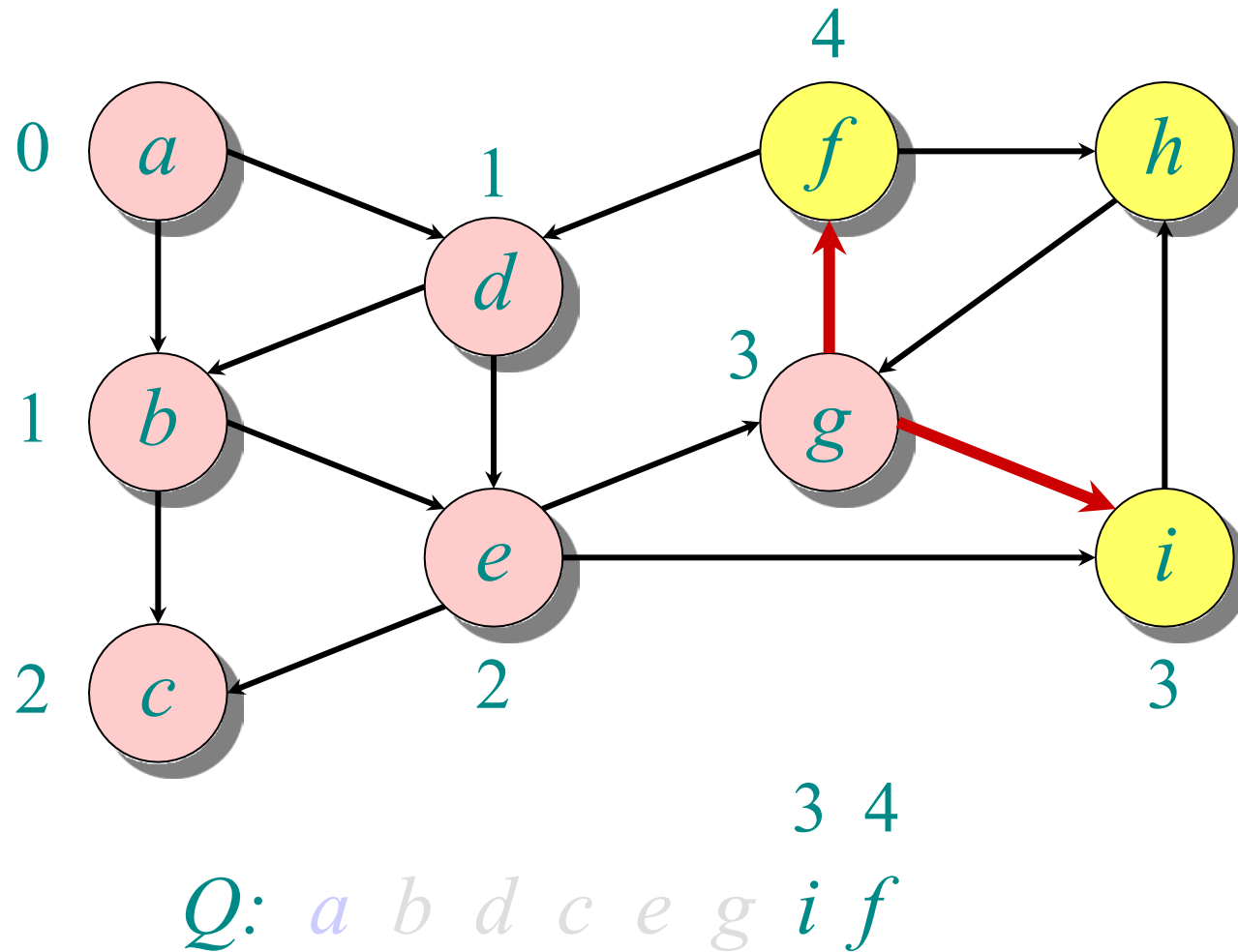
EXAMPLE OF BREADTH-FIRST SEARCH



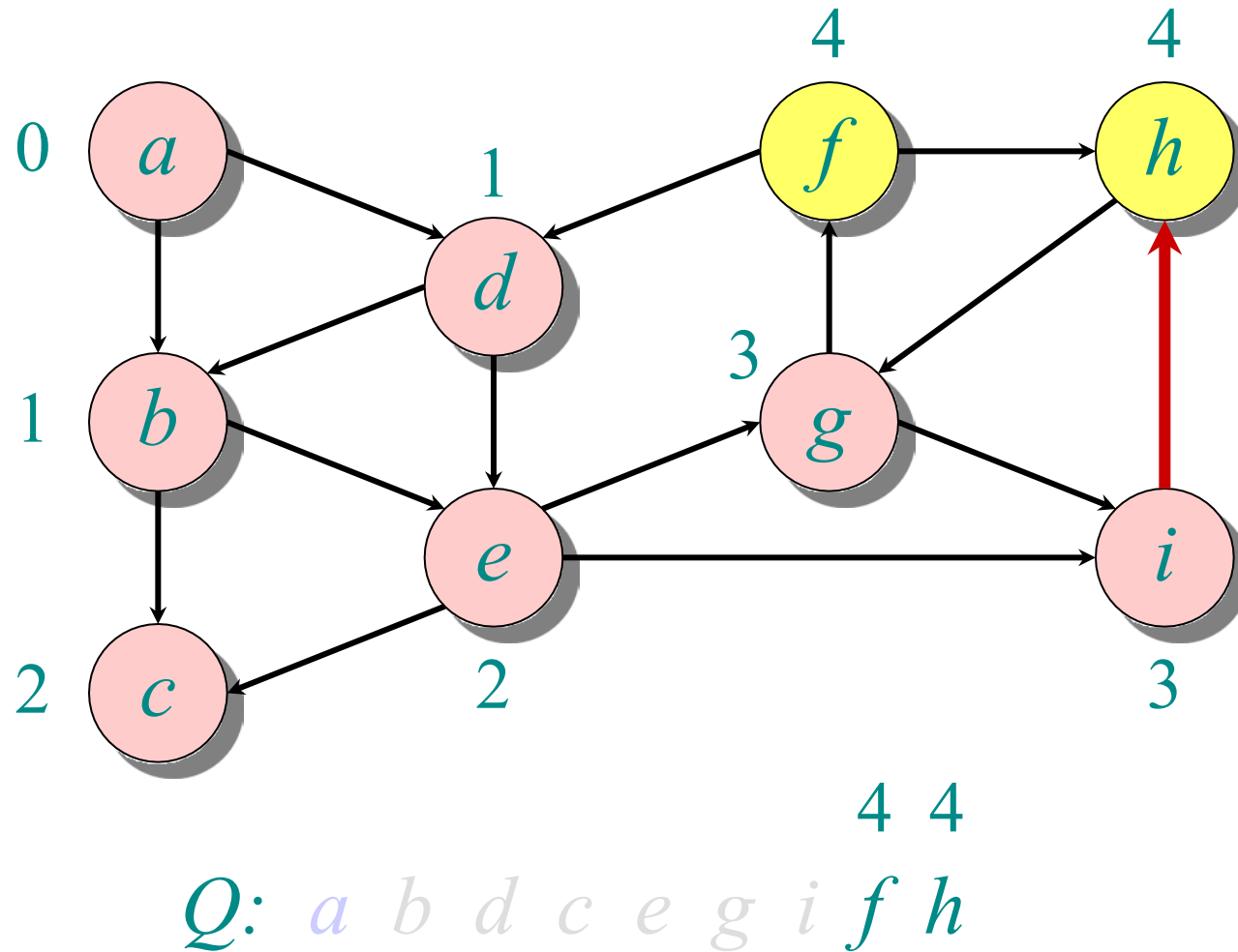
EXAMPLE OF BREADTH-FIRST SEARCH



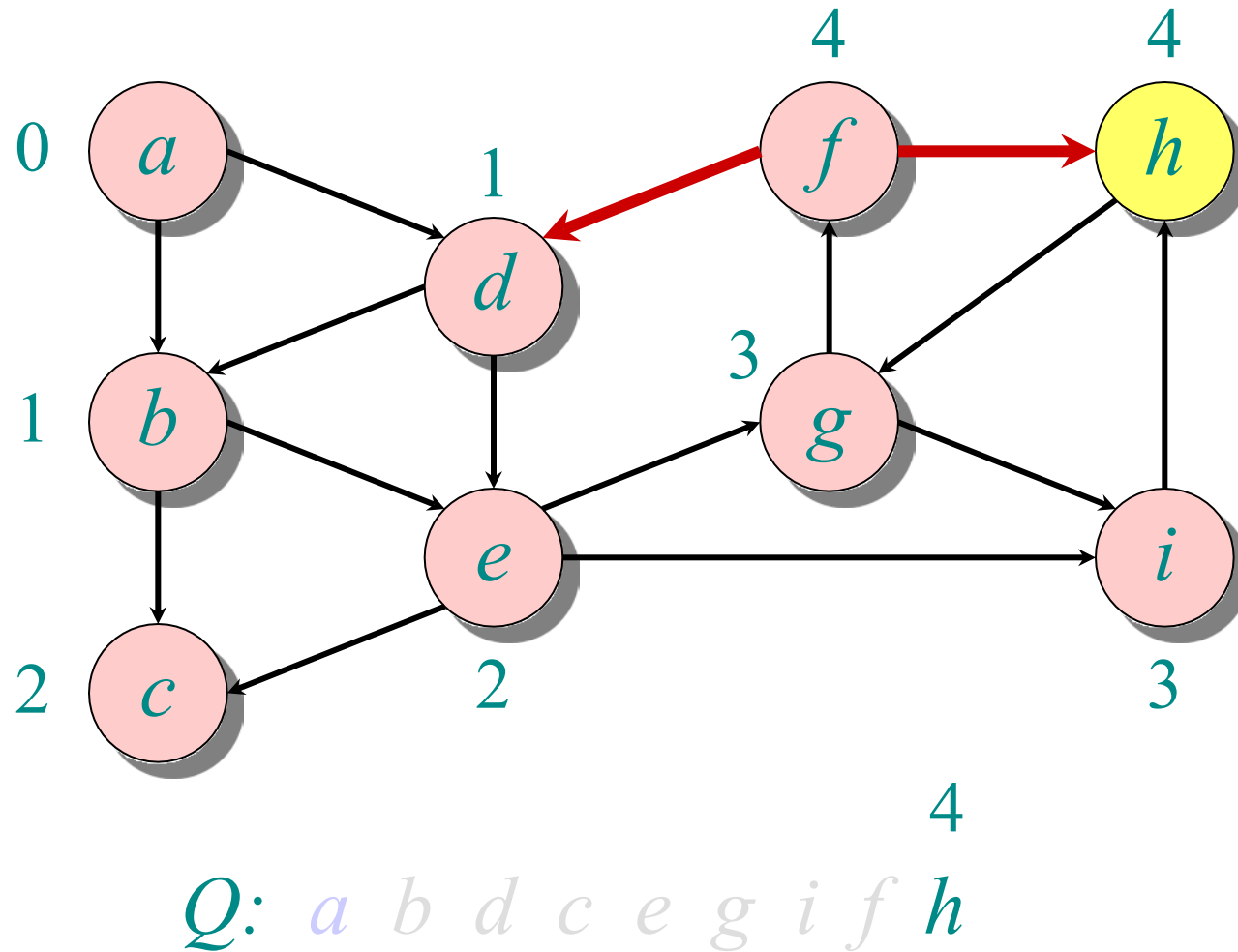
EXAMPLE OF BREADTH-FIRST SEARCH



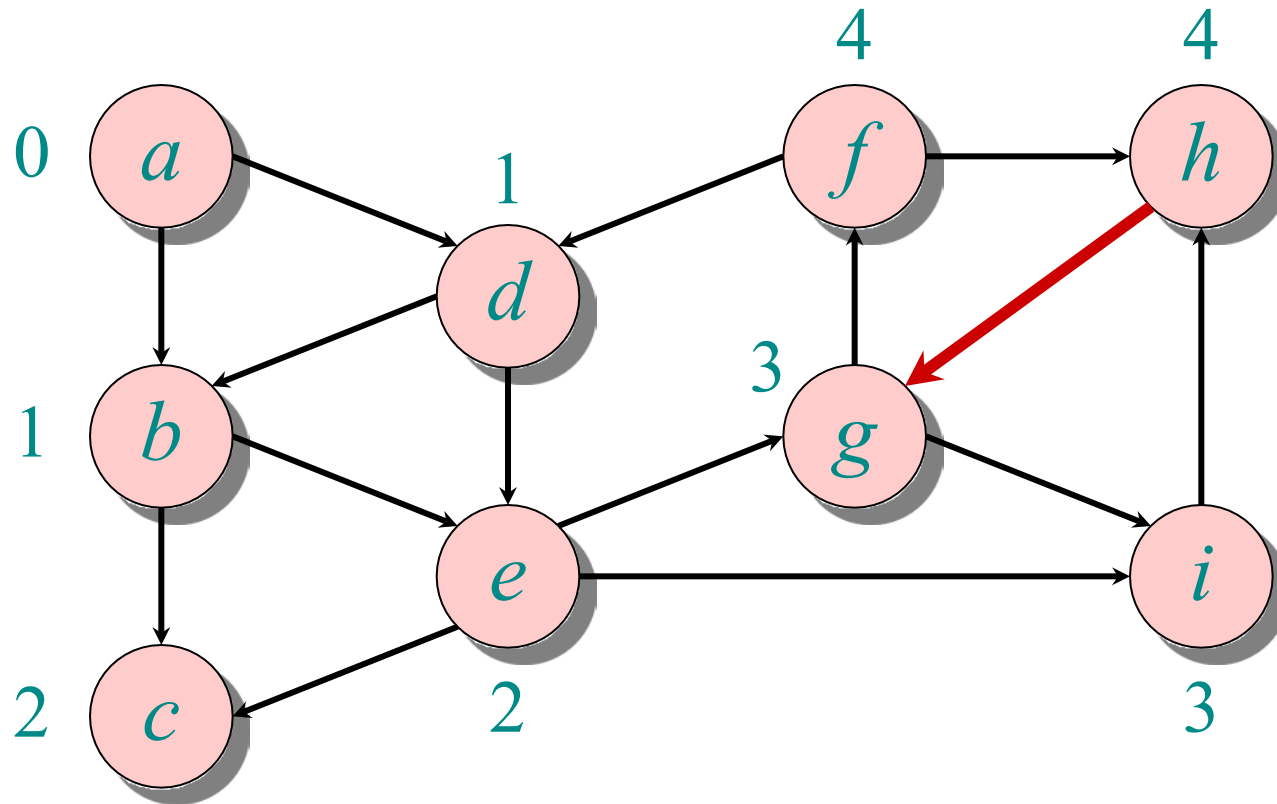
EXAMPLE OF BREADTH-FIRST SEARCH



EXAMPLE OF BREADTH-FIRST SEARCH

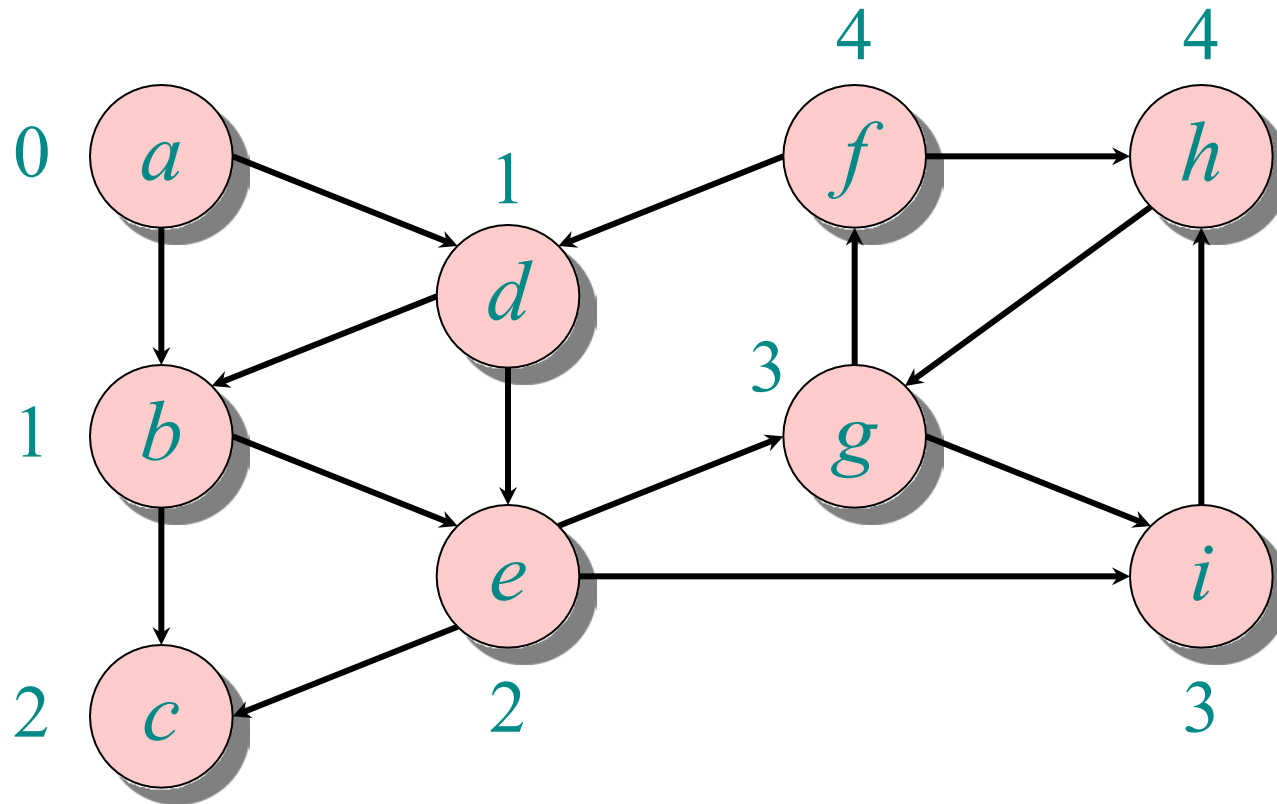


EXAMPLE OF BREADTH-FIRST SEARCH



Q: *a* *b* *d* *c* *e* *g* *i* *f* *h*

EXAMPLE OF BREADTH-FIRST SEARCH



Q: *a* *b* *d* *c* *e* *g* *i* *f* *h*

REFERENCE

○ Introduction to Algorithms

- Single Source Shortest Path
- Chapter # 24
- Thomas H. Cormen